

DATA STRUCTURES AND ALGORITHMS

Heaps

CCE 105
Data
Structures

Prepared by:



01

Heaps

Heaps

A heap is a binary tree with these characteristics:

- **It's complete.**

This means it's completely filled in, reading from left to right across each row, although the last row need not be full.

- **It's (usually) implemented as an array.**
- **Each node in a heap satisfies the heap condition, which states that every node's key is larger/smaller than the keys of its children.**

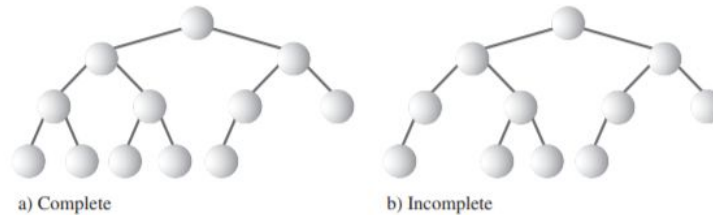


FIGURE 12.1 Complete and incomplete binary trees.



Heaps

Heaps are mostly used to implement priority queues. However, there's a very close relationship between a priority queue and the heap used to implement it.

Maxheap: $\text{key}(v) \leq \text{key}(\text{parent}(v))$

Minheap: $\text{key}(v) \geq \text{key}(\text{parent}(v))$



Heaps

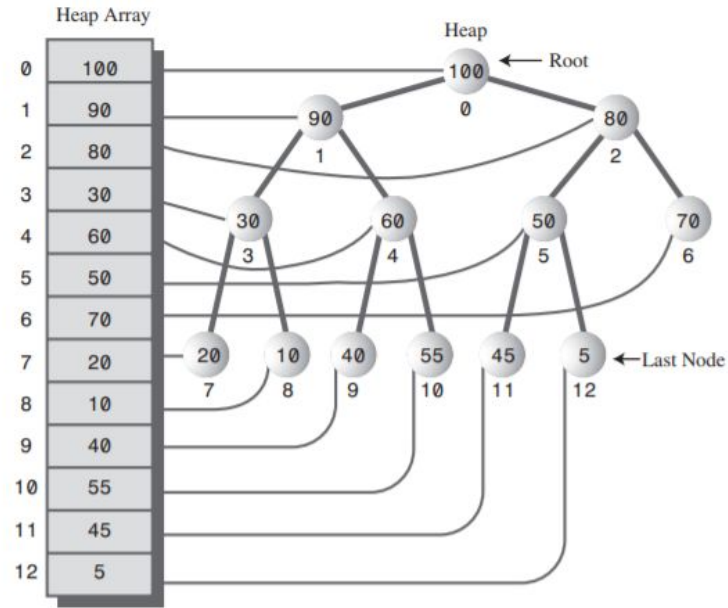


FIGURE 12.2 A heap and its underlying array.

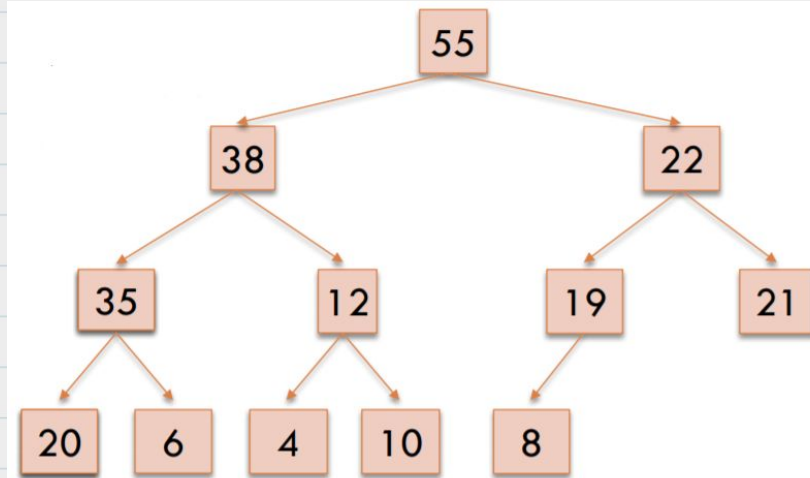


Heaps

Completeness Property

Every level (except last)
completely filled.

Nodes on bottom level are
as far left as possible.

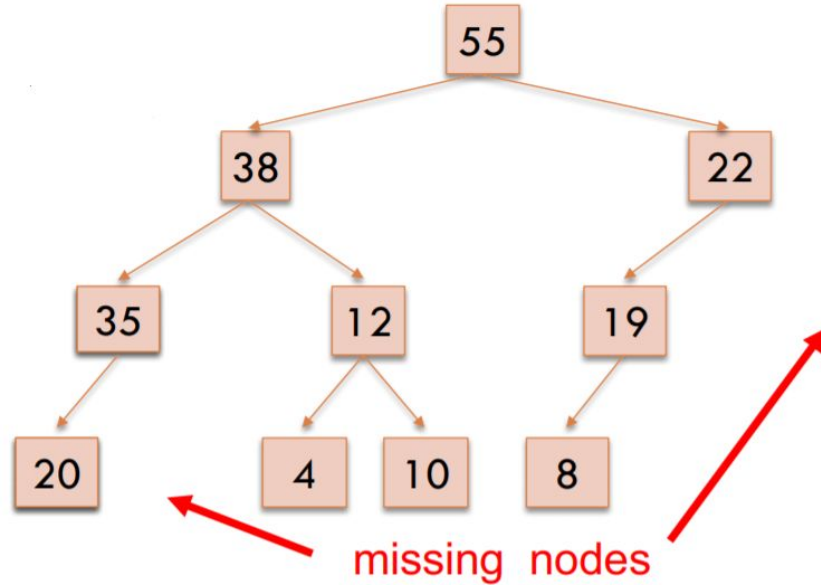


Heaps

Completeness Property

Not a heap because:

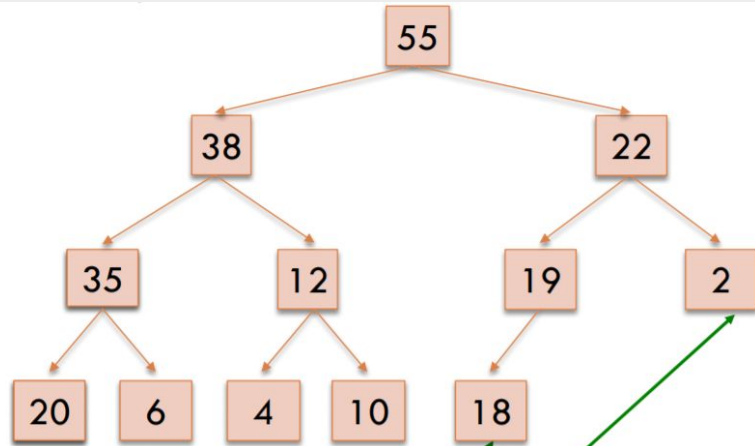
- missing a node on level 2
- bottom level nodes are not as far left as possible



Heaps

Order Property

Every element is $\leq$ its parent



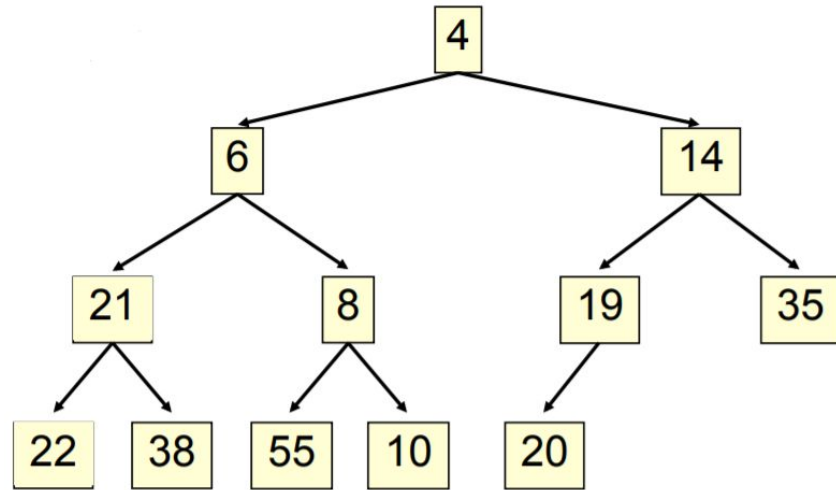
Note: Bigger elements
can be deeper in the tree!



Heaps

Order Property

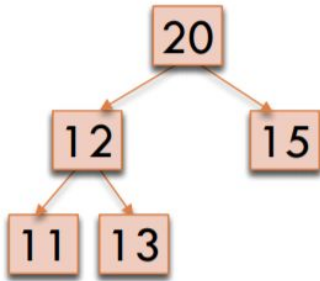
Every element is $\geq$ its parent



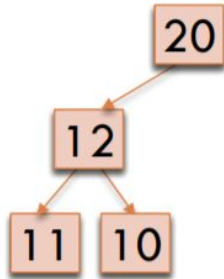
Heaps

Order Property

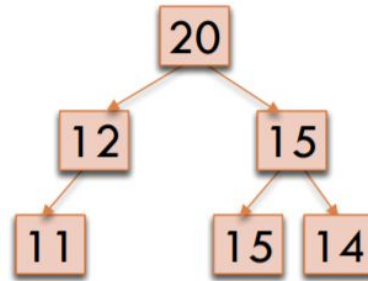
Which of the following are valid heaps?



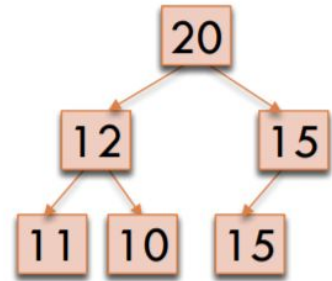
(a)



(b)



(c)



(d)

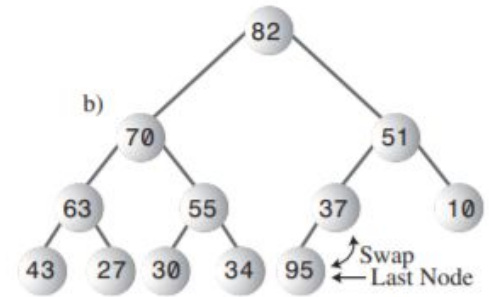
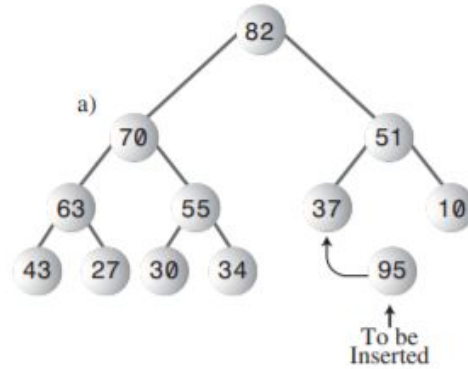


Heaps

Insertion

Inserting a node is also easy. Insertion uses trickle up, rather than trickle down.

Initially, the node to be inserted is placed in the first open position at the end of the array, increasing the array size by one

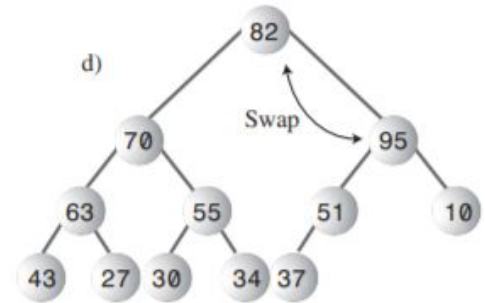
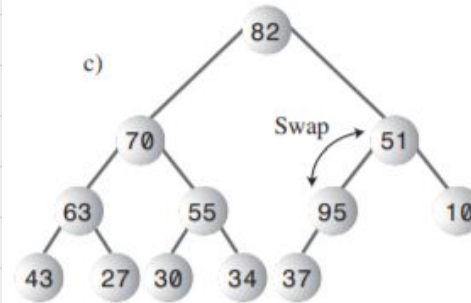


Heaps

Insertion

Inserting a node is also easy. Insertion uses trickle up, rather than trickle down.

Initially, the node to be inserted is placed in the first open position at the end of the array, increasing the array size by one

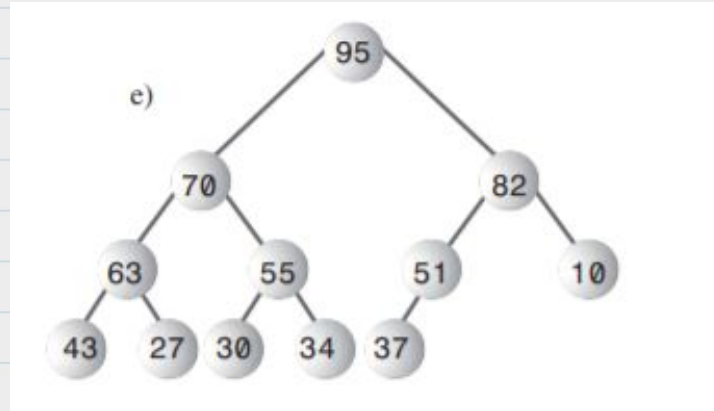


Heaps

Insertion

Inserting a node is also easy. Insertion uses trickle up, rather than trickle down.

Initially, the node to be inserted is placed in the first open position at the end of the array, increasing the array size by one

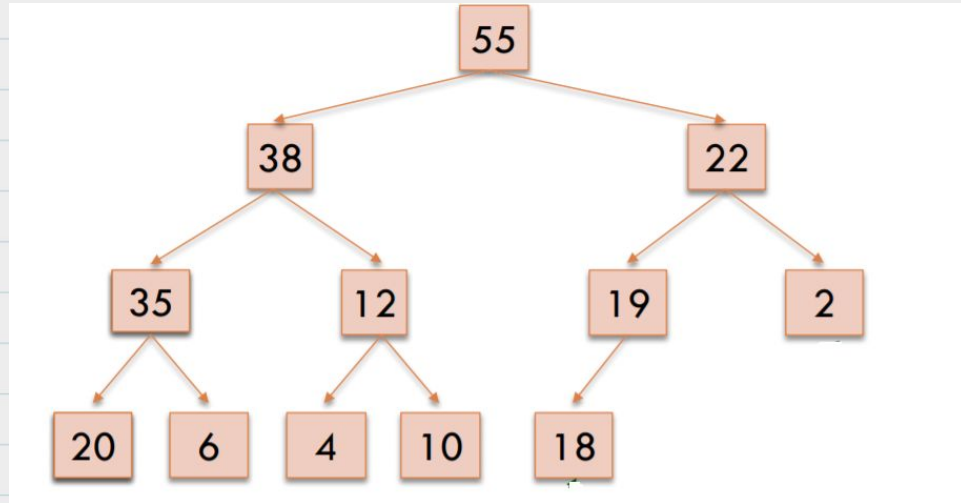


Heaps

Insertion

Maxheap

Let's try inserting 50

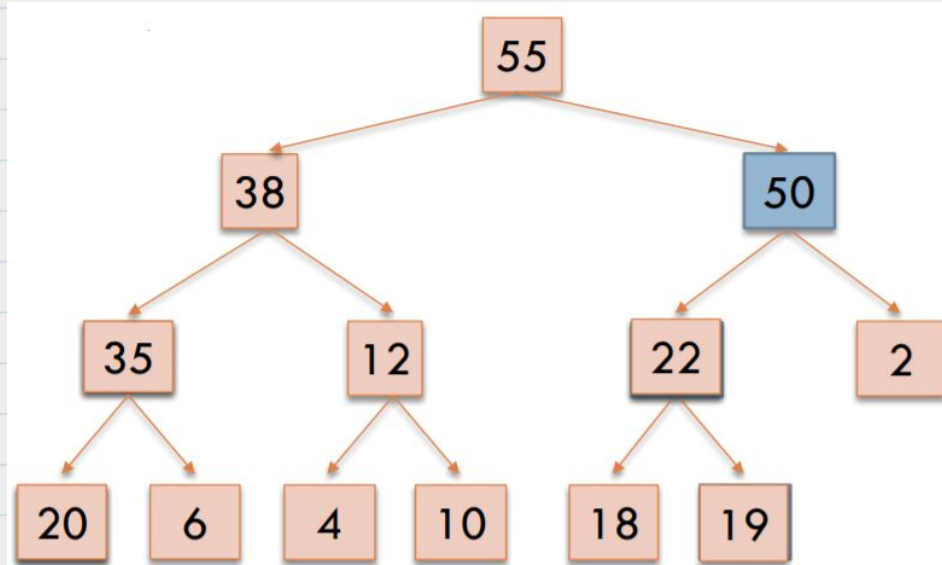


Heaps

Insertion

Maxheap

Let's try inserting 50



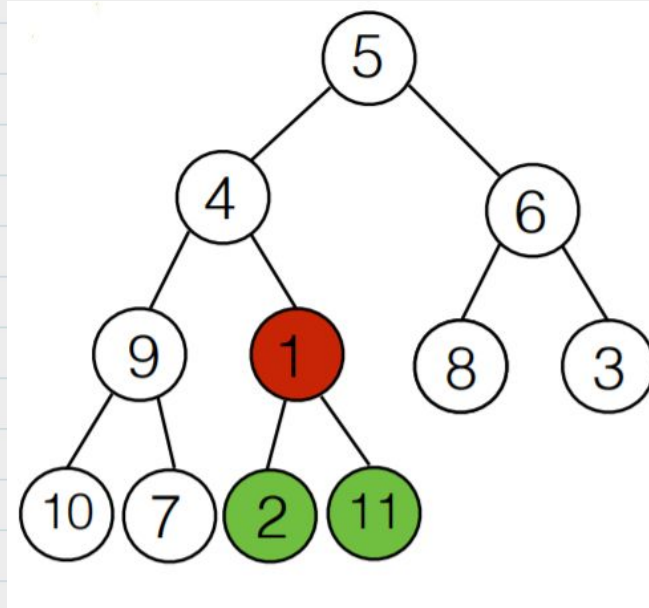
Heaps

Insertion

Minheap

Insert the following elements:

5 4 6 9 1 8 3 10 7 2 11



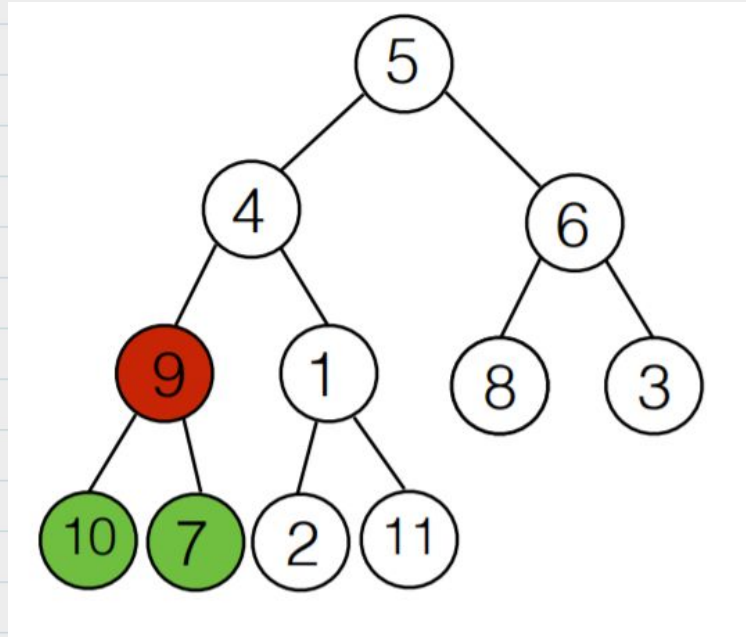
Heaps

Insertion

Minheap

Insert the following elements:

5 4 6 9 1 8 3 10 7 2 11



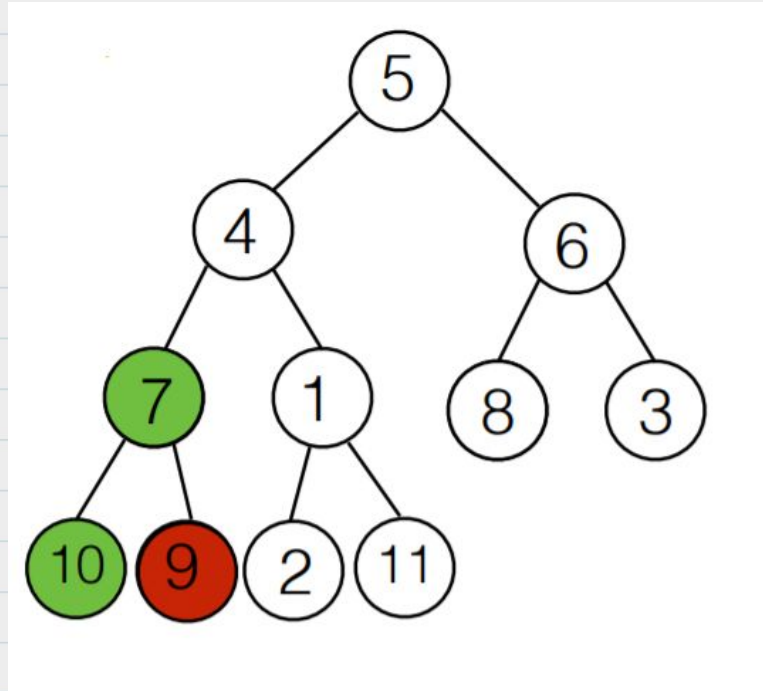
Heaps

Insertion

Minheap

Insert the following elements:

5 4 6 9 1 8 3 10 7 2 11



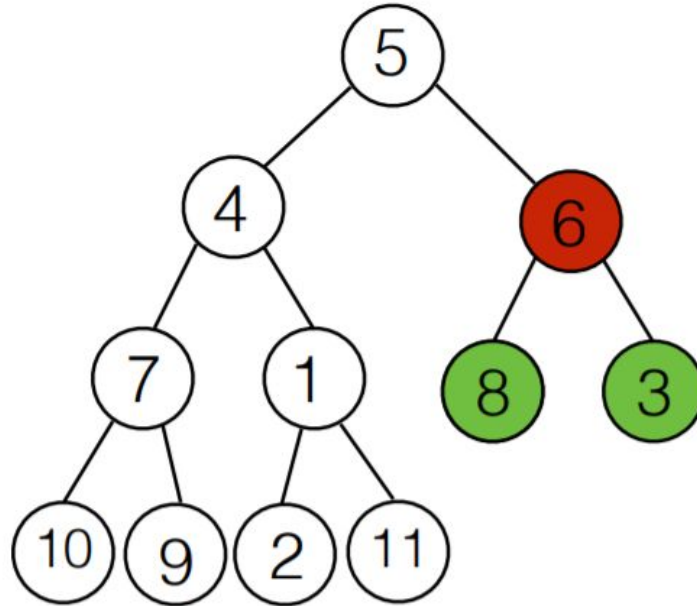
Heaps

Insertion

Minheap

Insert the following elements:

5 4 6 9 1 8 3 10 7 2 11



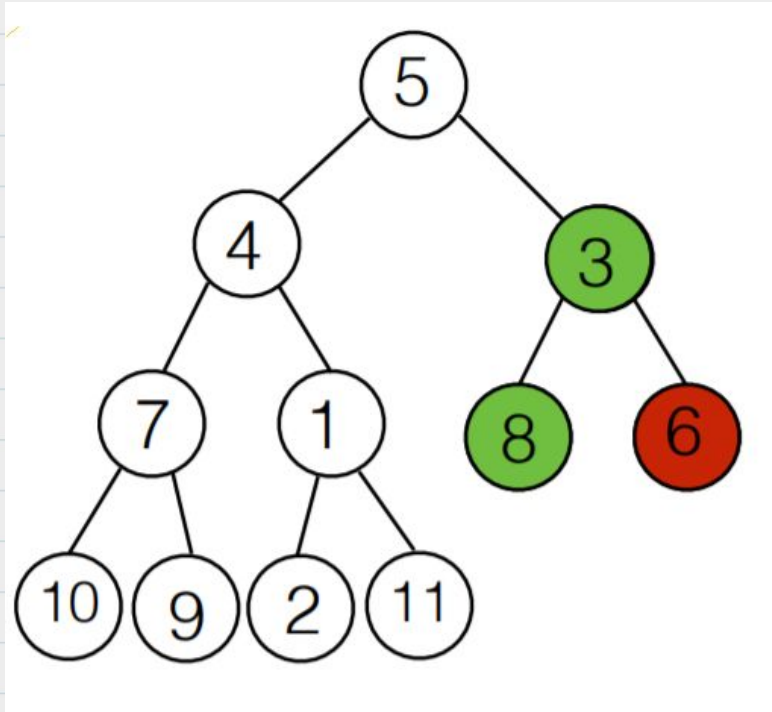
Heaps

Insertion

Minheap

Insert the following elements:

5 4 6 9 1 8 3 10 7 2 11



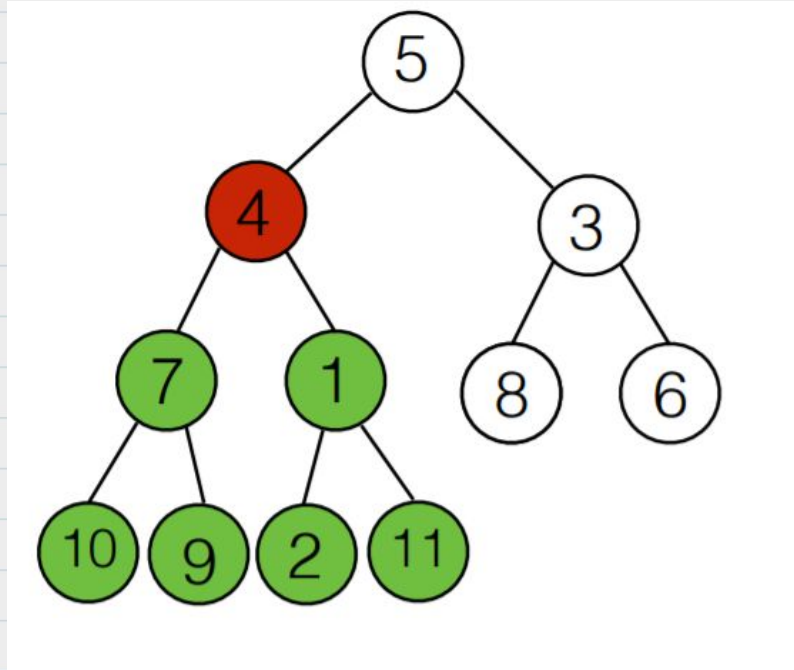
Heaps

Insertion

Minheap

Insert the following elements:

5 4 6 9 1 8 3 10 7 2 11



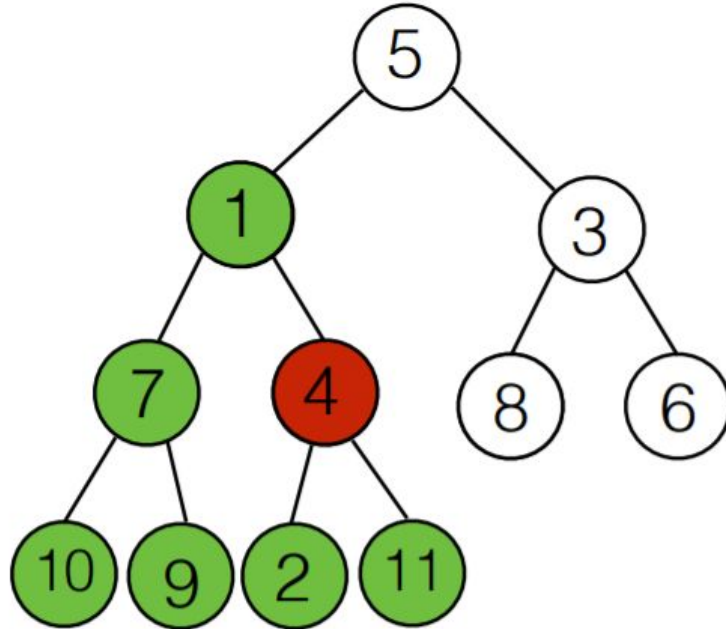
Heaps

Insertion

Minheap

Insert the following elements:

5 4 6 9 1 8 3 10 7 2 11



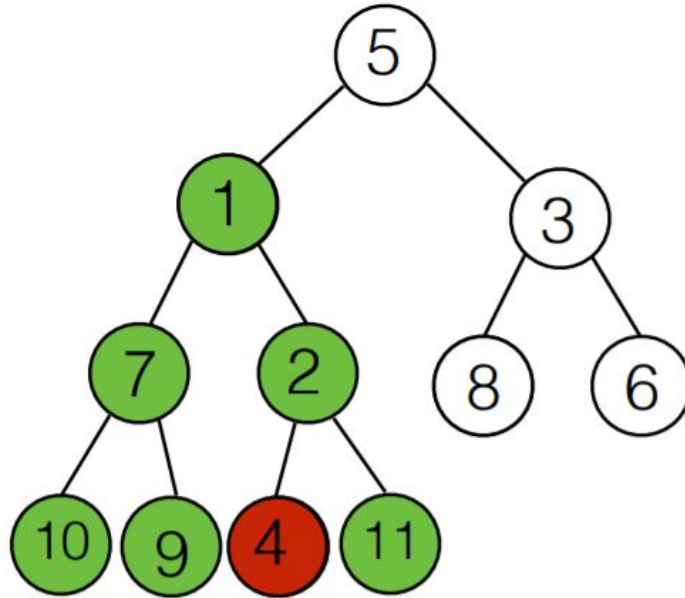
Heaps

Insertion

Minheap

Insert the following elements:

5 4 6 9 1 8 3 10 7 2 11



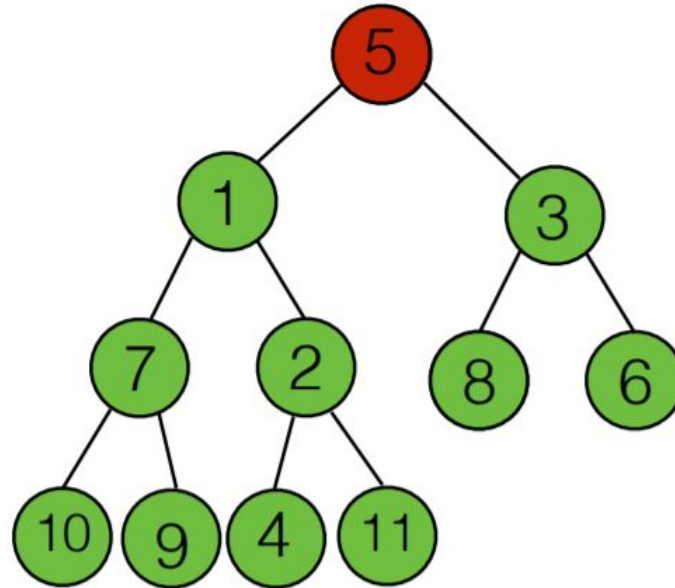
Heaps

Insertion

Minheap

Insert the following elements:

5 4 6 9 1 8 3 10 7 2 11



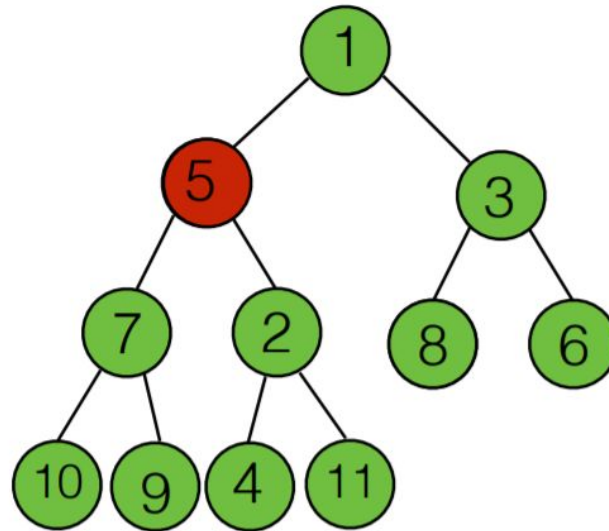
Heaps

Insertion

Minheap

Insert the following elements:

5 4 6 9 1 8 3 10 7 2 11



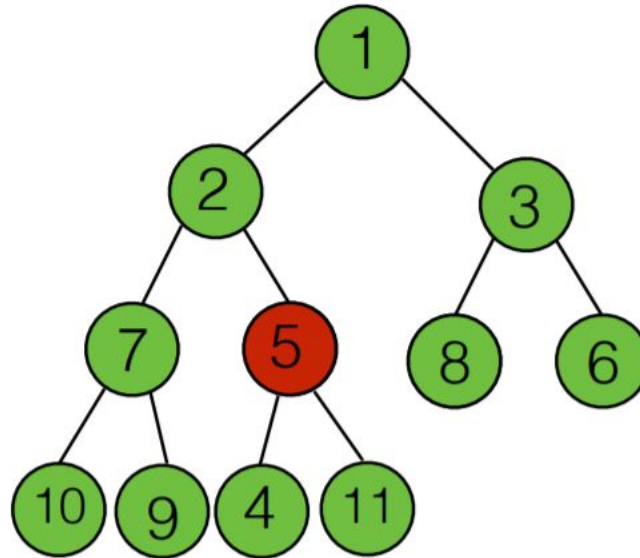
Heaps

Insertion

Minheap

Insert the following elements:

5 4 6 9 1 8 3 10 7 2 11



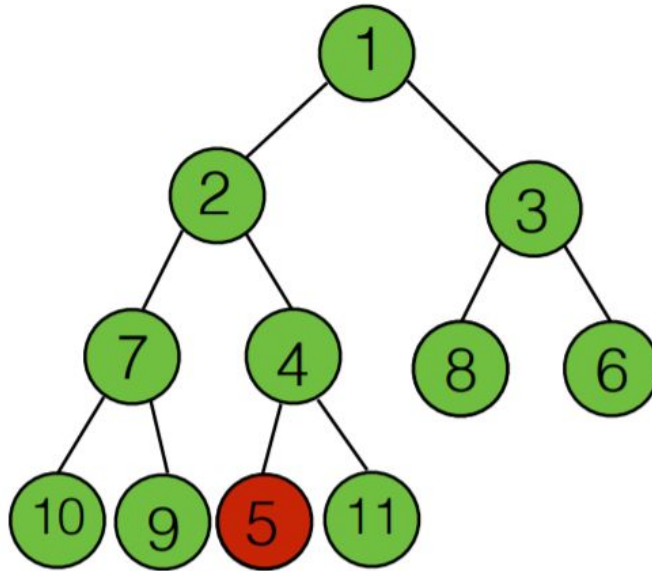
Heaps

Insertion

Minheap

Insert the following elements:

5 4 6 9 1 8 3 10 7 2 11



Heaps

Removal

Removal means removing the node with the maximum/minimum key. This node is always the root, so removing it is easy. The root is always at index 0 of the heap array.

```
maxNode = heapArray[0];
```

The problem is that once the root is gone, the tree is no longer complete; there's an empty cell. This "hole" must be filled in. We could shift all the elements in the array down one cell, but there's a much faster approach.

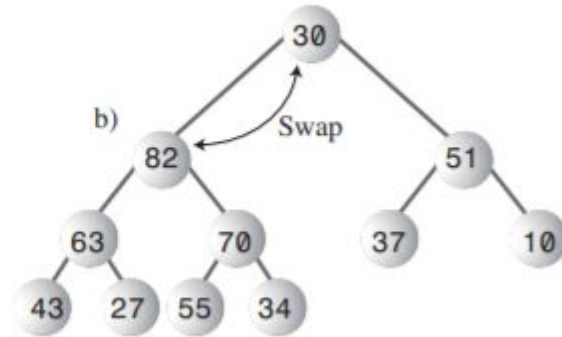
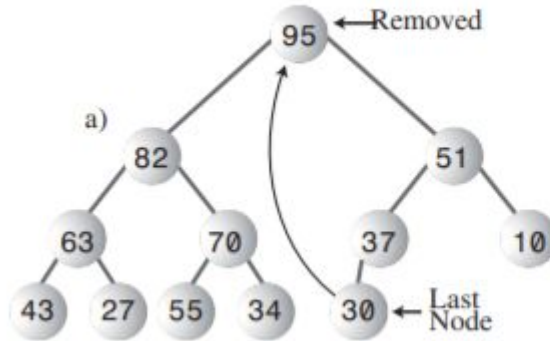


Heaps

Removal

Here are the steps for removing the maximum node:

1. Remove the root.
2. Move the last node into the root.
3. Trickle the last node down until it's below a larger node and above a smaller one.

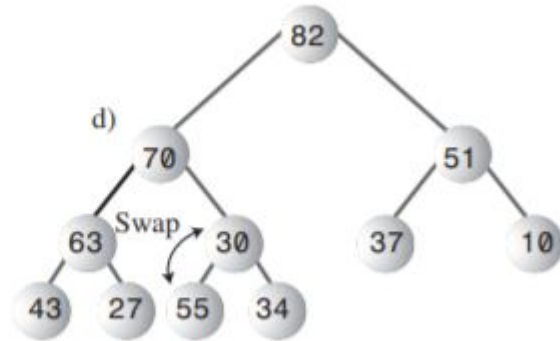
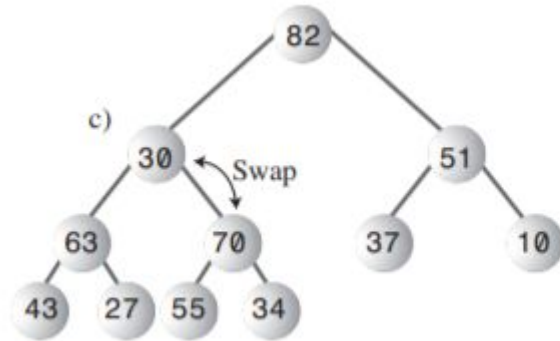


Heaps

Removal

Here are the steps for removing the maximum node:

1. Remove the root.
2. Move the last node into the root.
3. Trickle the last node down until it's below a larger node and above a smaller one.

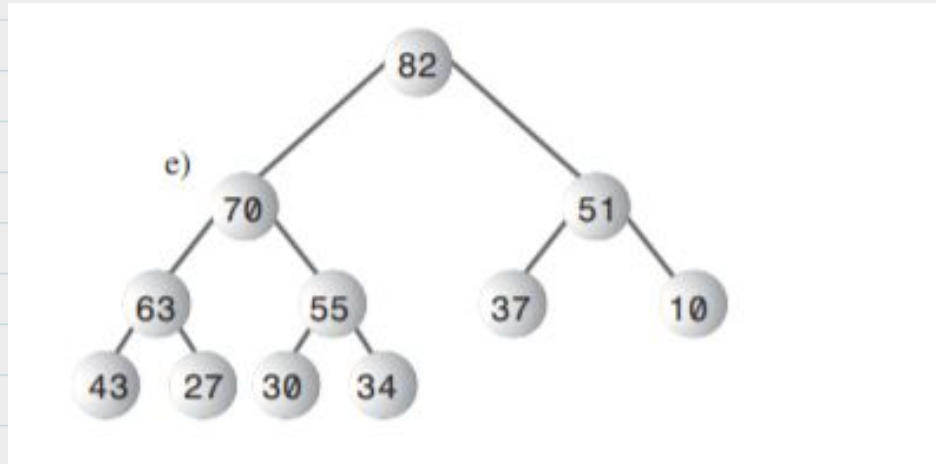


Heaps

Removal

Here are the steps for removing the maximum node:

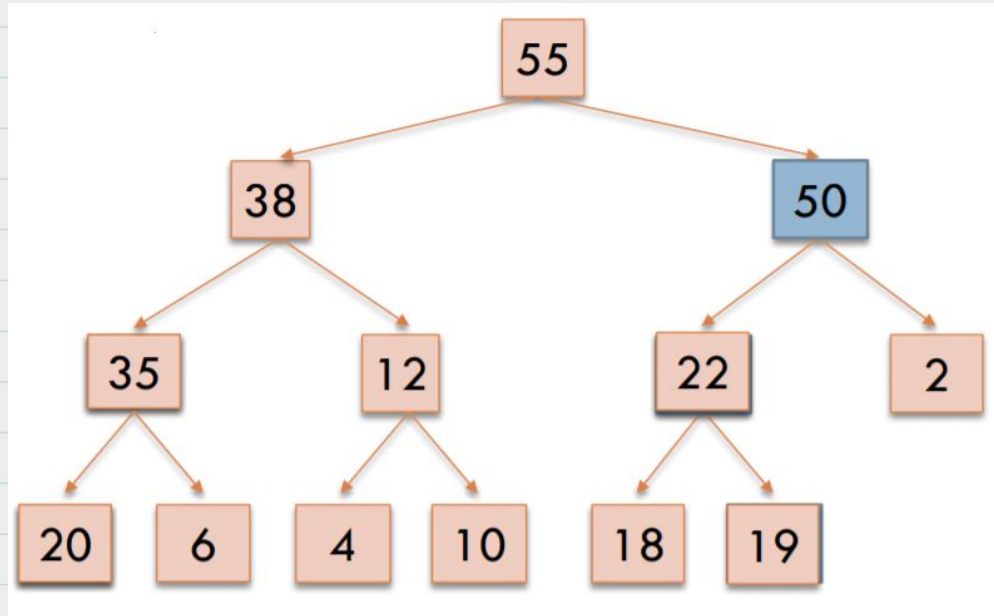
1. Remove the root.
2. Move the last node into the root.
3. Trickle the last node down until it's below a larger node and above a smaller one.



Heaps

Removal

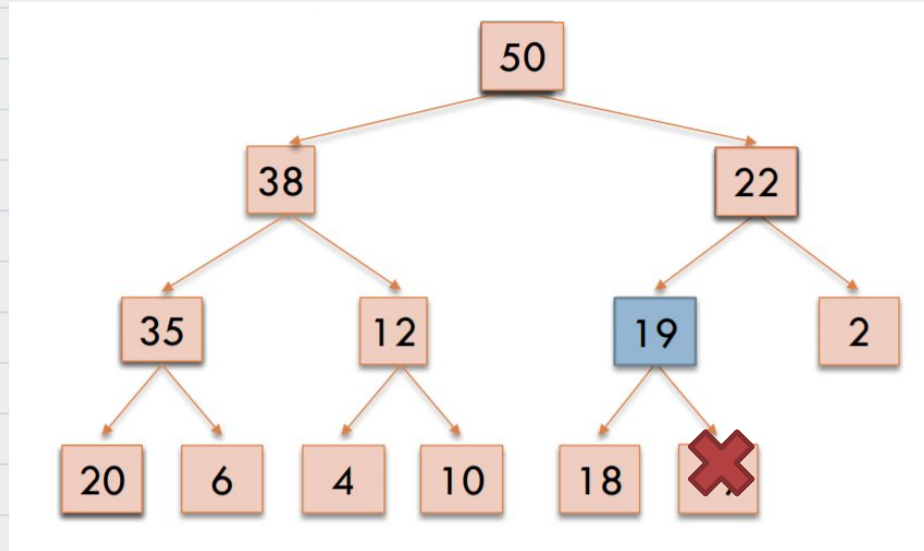
Let's try deleting 55



Heaps

Removal

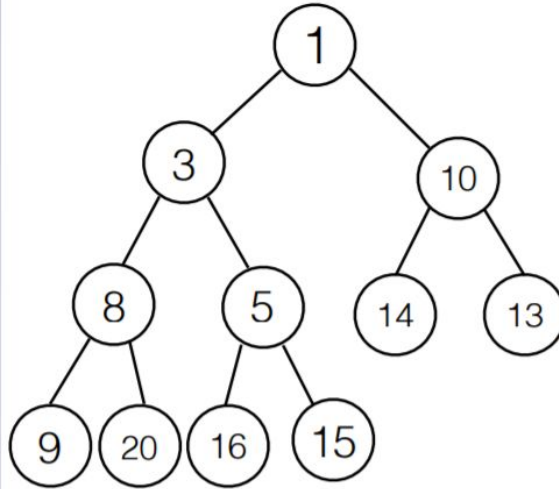
Let's try deleting 55



Heaps

Removal

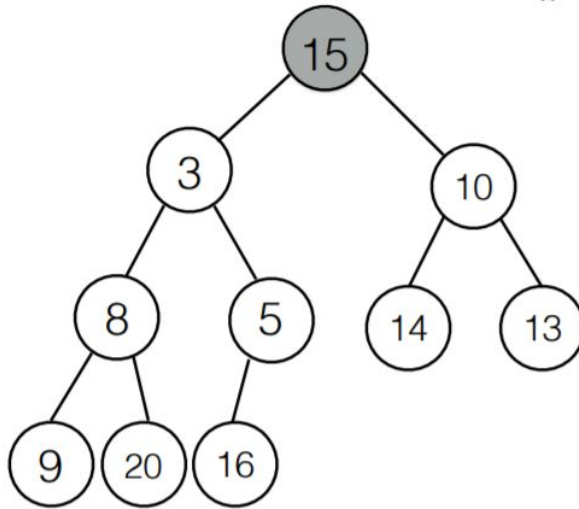
Let's try deleting 1



Heaps

Removal

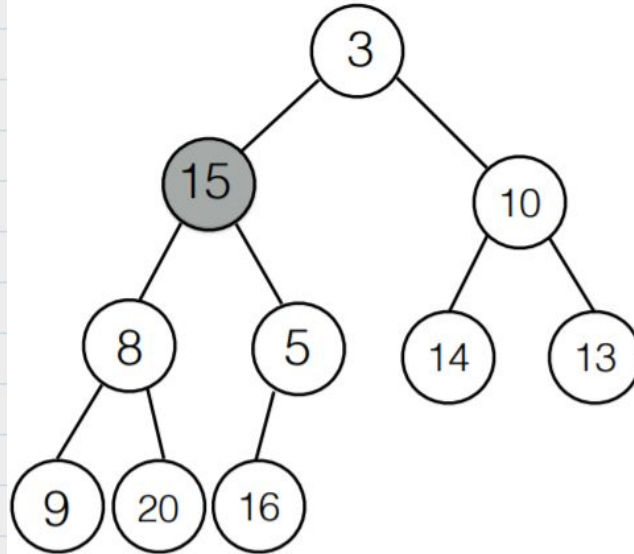
Let's try deleting 1



Heaps

Removal

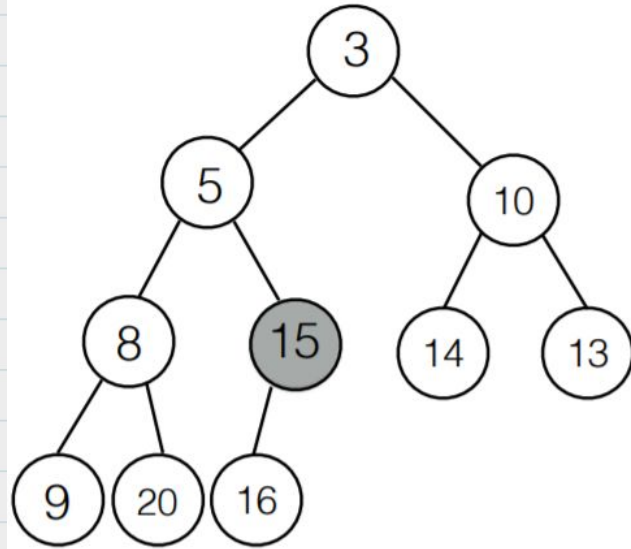
Let's try deleting 1



Heaps

Removal

Let's try deleting 1

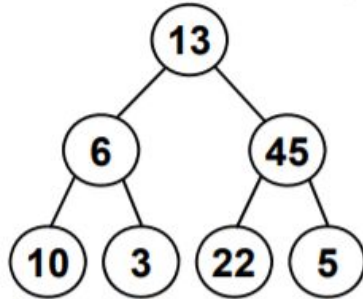


Heaps

Heapsort

- Sort the following array:

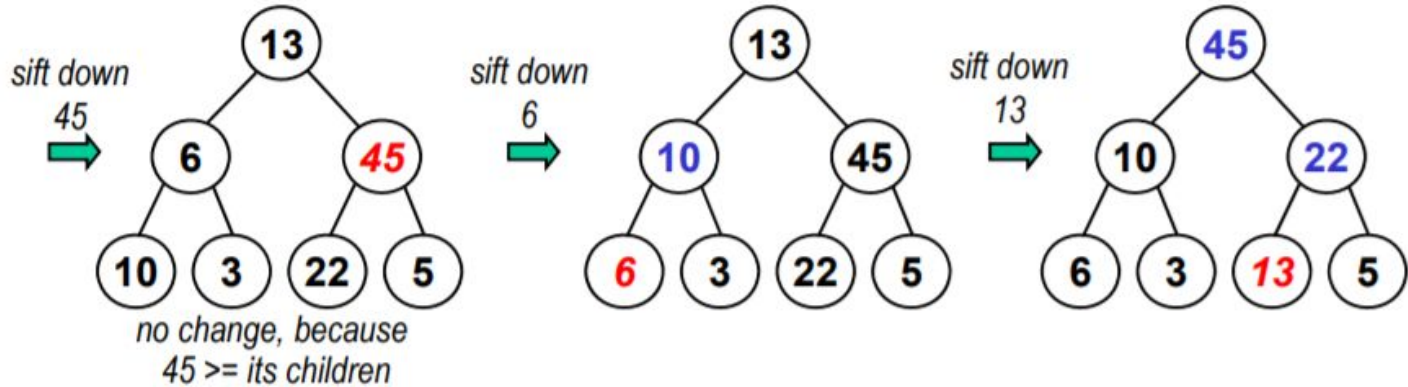
0	1	2	3	4	5	6
13	6	45	10	3	22	5
- Here's the corresponding complete tree:



Heaps

Heapsort

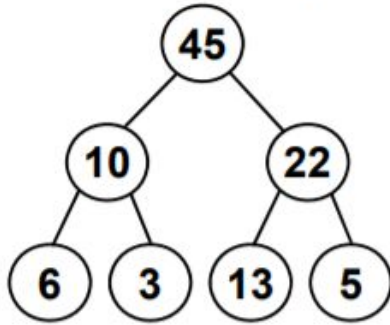
- Begin by converting it to a heap:



Heaps

Heapsort

- Here's the heap in both tree and array forms:



0	1	2	3	4	5	6
45	10	22	6	3	13	5

endUnsorted: 6

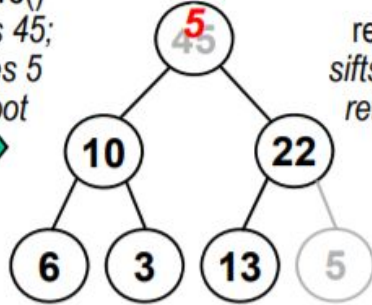


Heaps

Heapsort

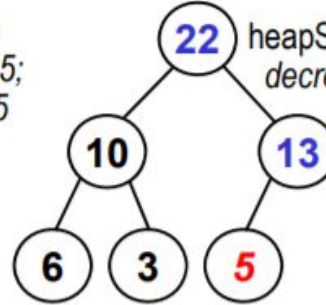
- Remove the largest item and put it in place:

remove()
copies 45;
moves 5
to root



toRemove: 45

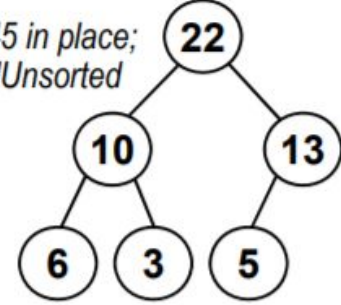
remove()
sifts down 5;
returns 45



0	1	2	3	4	5	6
22	10	13	6	3	5	

endUnsorted: 6
largestRemaining: 45

heapSort() puts 45 in place;
decrements endUnsorted



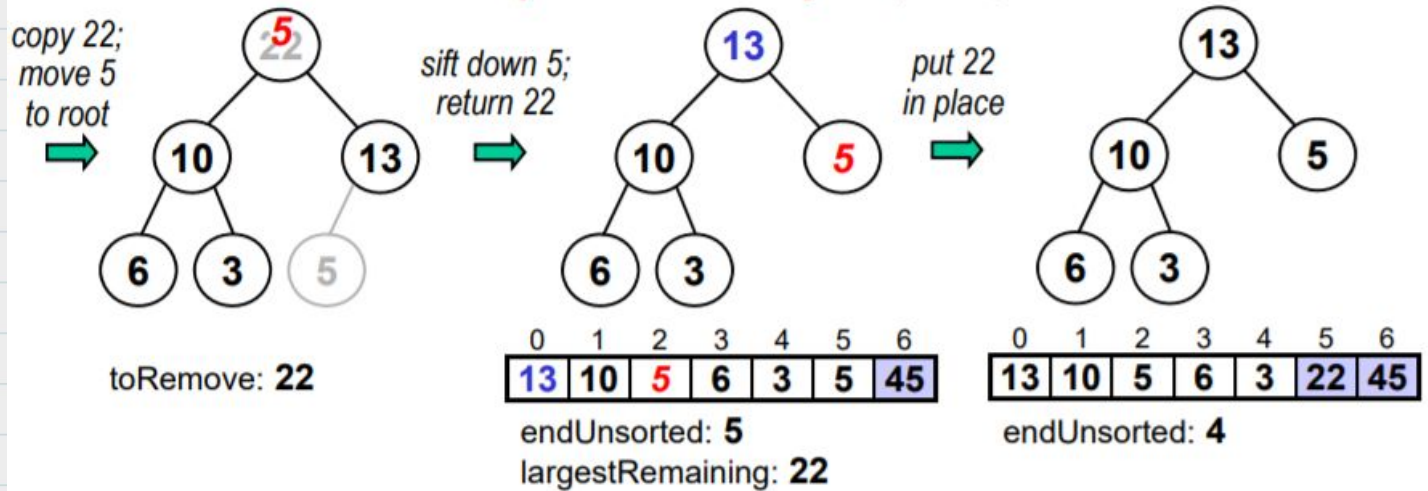
0	1	2	3	4	5	6
22	10	13	6	3	5	45

endUnsorted: 5



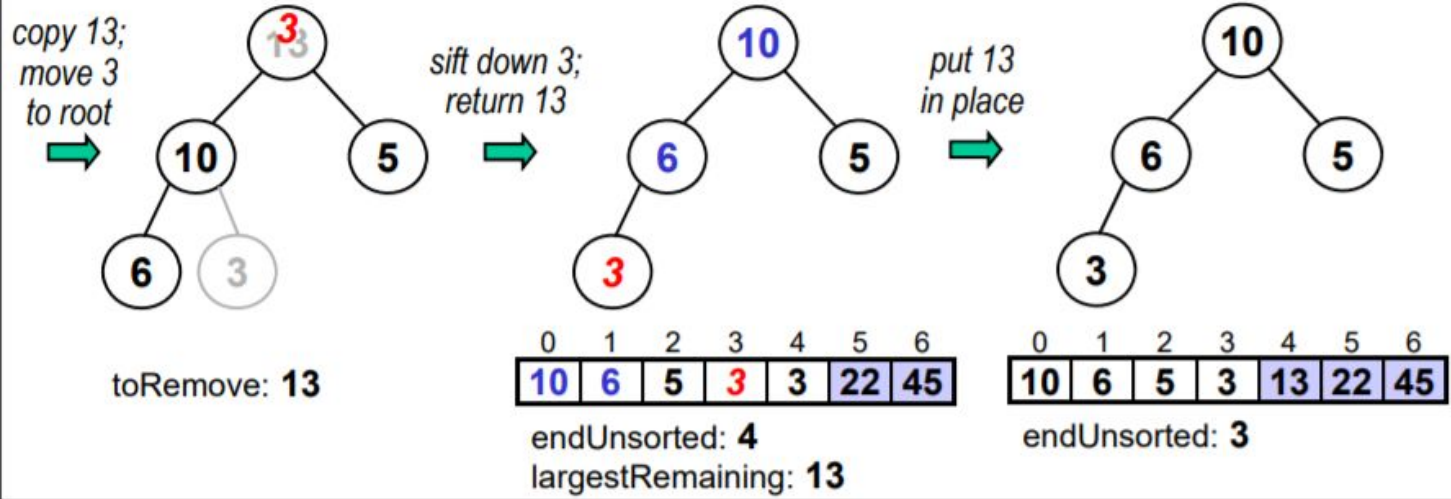
Heaps

Heapsort



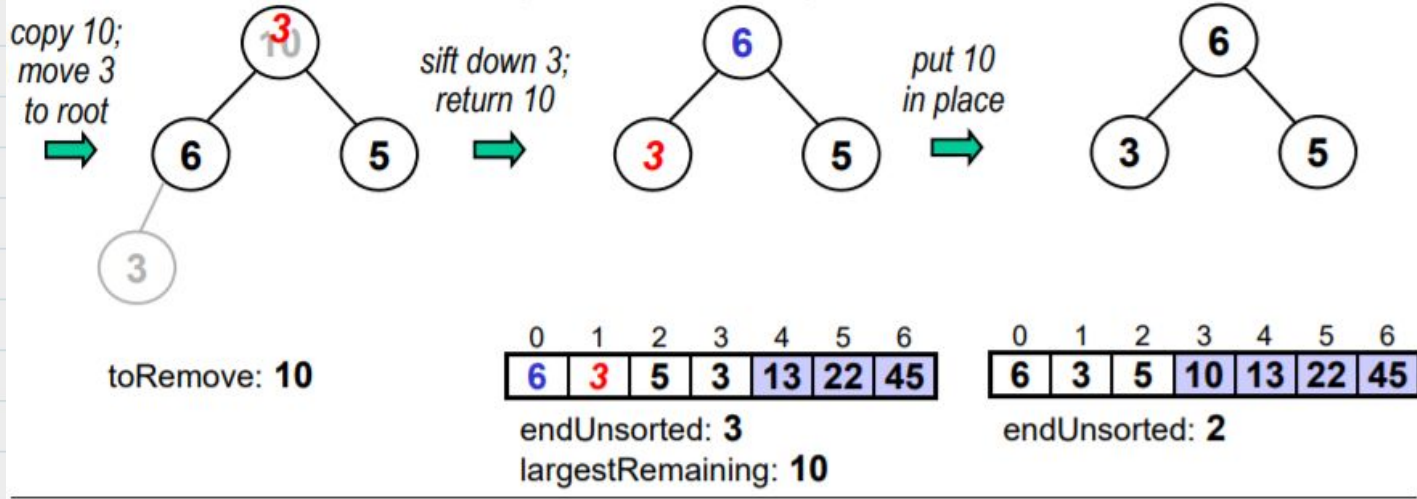
Heaps

Heapsort



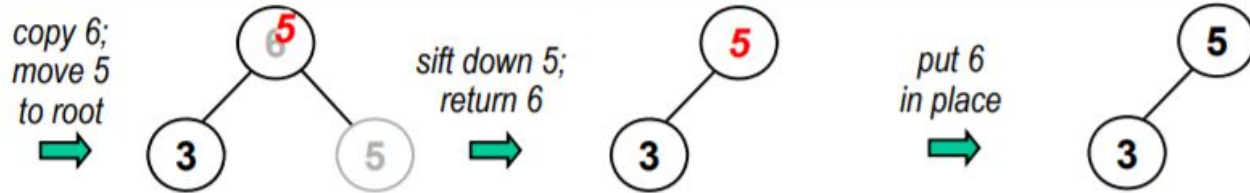
Heaps

Heapsort



Heaps

Heapsort



toRemove: 6

0	1	2	3	4	5	6
5	3	5	10	13	22	45

endUnsorted: 2

largestRemaining: 6

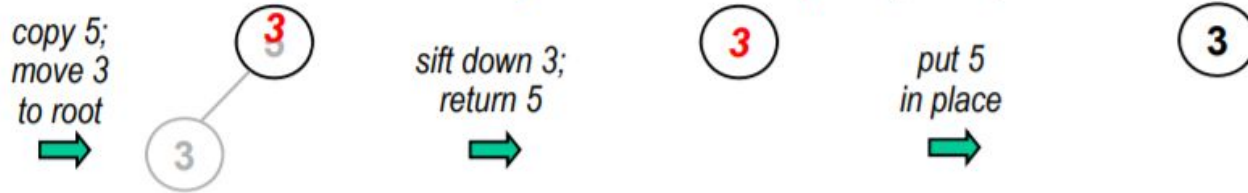
0	1	2	3	4	5	6
5	3	6	10	13	22	45

endUnsorted: 1



Heaps

Heapsort



toRemove: 5

0	1	2	3	4	5	6
3	3	6	10	13	22	45

endUnsorted: 1

largestRemaining: 5

0	1	2	3	4	5	6
3	5	6	10	13	22	45

endUnsorted: 0



Heaps

Let us try!

35 33 42 10 14 19 27 44 26

31

1. Create a heap tree.
2. Create a Maxheap.
3. Delete the largest element in Maxheap.
4. Sort using heapsort (maxheap).

